```python
"""

Main application controller for the Weather Dashboard.

Menu-driven CLI that ties together all modules:
  api_client  -> fetch geo-coding & weather data
  storage     -> persist favorites and log
  display     -> render panels, charts, tables
  charts      -> generate matplotlib PNG charts

"""

# Standard library imports  Modules and packages
import sys
from datetime import date, datetime

# Local module imports
from api_client import GeocodingClient, WeatherClient
from storage import (
    load_favorites,
    add_favorite,
    remove_favorite,
    append_to_log,
    load_log,
)
from display import (
    print_current_weather,
    print_forecast_table,
    print_temperature_chart,
    print_favorites,
    print_log_summary,
)
from charts import generate_all_charts
from exceptions import (
    WeatherDashboardError,
    APIError,
    CityNotFoundError,
    FileStorageError,
    InvalidInputError,
)
```

```python

# Instantiate service objects  Classes topic

geocoder = GeocodingClient()
weather_client = WeatherClient()



# Helper functions  Functions topic


def banner() -> None:
    """Print the application header."""
    print()
    print("=" * 48)
    print("         Weather Dashboard")
    print("      M602 Computer Programming")
    print("=" * 48)
    now: datetime = datetime.now()
    print(f"  Today is {now.strftime('%A, %B %d %Y - %H:%M')}")
    print()


def show_menu() -> None:
    """Display the main menu options."""
    print("")
    print("          MAIN MENU                  ")
    print("                                     ")
    print("  1. Search weather for a city       ")
    print("  2. View favorites                  ")
    print("  3. Add city to favorites           ")
    print("  4. Remove city from favorites      ")
    print("  5. Show weather log summary        ")
    print("  6. Exit                            ")
    print("")


def get_menu_choice() -> int:
    """
    Prompt user for a menu choice and validate it.
```

```python

    Returns:
        int: valid choice in range 1-6.

    Raises:
        InvalidInputError: if input cannot be parsed or is out of range.
    """
    raw: str = input("  Enter choice (1-6): ").strip()
    if not raw.isdigit():
        raise InvalidInputError(f"'{raw}' is not a valid number.")
    choice: int = int(raw)
    if not (1 <= choice <= 6):
        raise InvalidInputError(f"Choice must be between 1 and 6, got {choice}.")
    return choice


def search_city_weather() -> None:
    """
    Prompt for a city, fetch weather, display results,
    generate matplotlib charts, and log the current record.

    """
    city_name: str = input("  Enter city name: ").strip()
    if not city_name:
        raise InvalidInputError("City name cannot be empty.")

    print(f"\n  Searching for '{city_name}'...")

    city = geocoder.search(city_name)

    print(f"  Found: {city.name}, {city.country}")
    lat, lon = city.coordinates
    print(f"  Coordinates: lat={lat}, lon={lon}\n")
```

```


    current, forecast = weather_client.fetch(city)


    print_current_weather(current)
    print_forecast_table(forecast)
    print_temperature_chart(forecast)


    summary: tuple = current.summary_tuple
    city_s, date_s, temp_s, desc_s = summary
    print(f"  Summary tuple -> city={city_s}, date={date_s}, "
          f"temp={temp_s}C, condition={desc_s}")
    print()


    print("  Generating charts...")
    try:
        path_temp, path_wind, path_dash = generate_all_charts(forecast, current)
        print(f"  Temperature chart : {path_temp}")
        print(f"  Wind chart        : {path_wind}")
        print(f"  Dashboard chart   : {path_dash}\n")
    except Exception as exc:
        print(f"  Warning: could not generate charts : {exc}\n")


    try:
        append_to_log(current)
        print("  Weather record saved to log.\n")
    except FileStorageError as exc:
        print(f"  Could not save log entry: {exc}\n")


    save_choice: str = input(
        "  Add this city to favorites? (y/n): "
```

```python
    ).strip().lower()
    if save_choice == "y":
        try:
            add_favorite(city)
            print(f"  '{city.name}' added to favorites.\n")
        except FileStorageError as exc:
            print(f"  Could not save favorite: {exc}\n")


def view_favorites() -> None:
    """Load and display saved favorite cities."""
    favorites = load_favorites()
    print_favorites(favorites)


def add_city_to_favorites() -> None:
    """Search for a city and add it to favorites."""
    city_name: str = input("  Enter city name to save: ").strip()
    if not city_name:
        raise InvalidInputError("City name cannot be empty.")

    city = geocoder.search(city_name)
    add_favorite(city)
    print(f"  '{city.name}, {city.country}' saved to favorites.\n")


def remove_city_from_favorites() -> None:
    """Remove a city from the favorites list."""
    favorites = load_favorites()
    if not favorites:
        print("  (no favorites to remove)\n")
        return

    print_favorites(favorites)
    city_name: str = input("  Enter city name to remove: ").strip()
```

```python
    remove_favorite(city_name)
    print(f"  Removed '{city_name}' from favorites.\n")


def show_log() -> None:
    """Display a summary of the weather history log."""
    rows: list[dict] = load_log()
    print_log_summary(rows)


# Main loop  Control statements topic


def main() -> None:
    """
    Application entry point.
    Runs the interactive menu loop.

    """
    banner()


    while True:
        show_menu()
        try:
            choice: int = get_menu_choice()
        except InvalidInputError as exc:
            print(f"\n  {exc}\n")
            continue


        if choice == 1:
            try:
                search_city_weather()
            except CityNotFoundError as exc:
                print(f"\n  {exc}\n")
            except APIError as exc:
                print(f"\n  {exc}\n")
            except InvalidInputError as exc:
```

```
                print(f"\n  {exc}\n")
            except FileStorageError as exc:
                print(f"\n  Storage issue: {exc}\n")

        elif choice == 2:
            try:
                view_favorites()
            except FileStorageError as exc:
                print(f"\n  {exc}\n")

        elif choice == 3:
            try:
                add_city_to_favorites()
            except (CityNotFoundError, APIError, FileStorageError, InvalidInputError) as exc:
                print(f"\n  {exc}\n")

        elif choice == 4:
            try:
                remove_city_from_favorites()
            except FileStorageError as exc:
                print(f"\n  {exc}\n")

        elif choice == 5:
            try:
                show_log()
            except FileStorageError as exc:
                print(f"\n  {exc}\n")

        elif choice == 6:
            print("\n  Goodbye!\n")
            sys.exit(0)


# Entry point guard  Modules and packages topic

if __name__ == "__main__":
    main()
```